CAHIER DES CHARGES

Projet de Fin d'Annee

Application de Gestion des Heures des Enseignants

Cas de l'Universite Virtuelle de Cote d'Ivoire (UVCI)

Etablissement	Universite Virtuelle de Cote d'Ivoire (UVCI)
Projet	Gestion des Heures des Enseignants
Technologies	Next.js | Python FastAPI | FaceAPI | SQLite
Equipe	5 membres
Duree	2 semaines
Date	Mars 2026

Version 1.0

Table des Matieres

1. Introduction

Ce document constitue le Cahier des Charges (CDC) du projet de fin d'annee intitule "Application de Gestion des Heures des Enseignants" developpe dans le cadre de l'Universite Virtuelle de Cote d'Ivoire (UVCI). Il definit de maniere exhaustive les besoins fonctionnels, les exigences techniques, l'architecture logicielle et le planning de realisation du projet.

L'objectif est de fournir une reference commune a toute l'equipe de developpement (5 membres) pour garantir la coherence, la qualite et la livraison dans les delais impartis de deux semaines.

2. Contexte et Problematique

2.1 Contexte

L'Universite Virtuelle de Cote d'Ivoire (UVCI) est un etablissement d'enseignement superieur dont le modele pedagogique repose sur la production et la diffusion de ressources numeriques. Les activites des enseignants vont au-dela du cours traditionnel : elles incluent la conception de ressources pedagogiques numeriques, la creation de sequences pedagogiques, l'elaboration de quiz et d'evaluations, la production d'activites interactives et la mise a jour reguliere des contenus.

2.2 Problematique

Actuellement, la gestion de ces activites repose sur des fichiers Excel et des procedures manuelles, ce qui engendre plusieurs dysfonctionnements majeurs :

- Erreurs frequentes dans le calcul des volumes horaires
- Absence de traçabilite des modifications apportees aux donnees
- Difficultes de consolidation et de centralisation des informations
- Retards dans la preparation des etats de paiement des enseignants
- Manque de visibilite globale sur la charge de travail des enseignants

Face a ces problemes, il est necessaire de developper une application web robuste, securisee et ergonomique pour automatiser et fiabiliser ces processus.

3. Objectifs du Projet

3.1 Objectif General

Developper une application web full-stack permettant la gestion automatisee et securisee des heures d'enseignement et des activites pedagogiques des enseignants de l'UVCI.

3.2 Objectifs Specifiques

- Permettre l'enregistrement et la gestion complete des enseignants

- Assurer la gestion des cours, des sequences et des ressources pedagogiques
- Calculer automatiquement les volumes horaires selon les baremes officiels
- Identifier et signaler les heures complementaires
- Securiser l'acces via une authentification combinant mot de passe et reconnaissance faciale (FaceAPI)
- Generer des tableaux de bord et des etats recapitulatifs exportables en PDF et Excel
- Fournir un espace personnel a chaque enseignant pour consulter ses activites

4. Perimetre Fonctionnel

4.1 Acteurs du Systeme

Le systeme implique trois types d'acteurs principaux, chacun disposant d'un niveau d'acces et de privileges specifiques :

Acteur	Role	Acces Principal
Administrateur	Gestion technique et configuration du systeme	Gestion comptes, parametrage, taux horaires, supervision
Secretaire Principal	Gestion administrative des enseignants	Saisie activites, generation etats, suivi volumes
Enseignant	Consultation de son espace personnel	Visualisation activites, heures, telechargement recapitulatif

4.2 Modules Fonctionnels

L'application est organisee autour des modules suivants :

Module 1 - Gestion des Enseignants

- Creation, modification et suppression d'un enseignant
- Consultation de la liste des enseignants avec filtres
- Informations gerees : nom, prenom, grade (Assistant / Maitre-Assistant / Professeur), statut (Permanent / Vacataire), departement, taux horaire, email, telephone

Module 2 - Gestion des Cours

- Gestion des cours : intitule, filiere, niveau (L1 a M2), semestre
- Nombre d'heures et credits associes au cours
- Association enseignant-cours

Module 3 - Gestion des Ressources Pedagogiques

- Enregistrement des ressources : contenus textuels, videos, documents, quiz, activites interactives, evaluations
- Structuration des cours en sequences pedagogiques (Niveau 1, 2 et 3)

- Gestion des types de ressources par niveau de complexite

Module 4 - Gestion des Activites Pedagogiques

- Saisie de la creation de ressources pedagogiques
- Saisie de la mise a jour de ressources existantes
- Association d'un niveau de complexite a chaque activite

Module 5 - Calcul Automatique des Volumes Horaires

Le systeme calcule automatiquement les volumes horaires selon la grille officielle de l'UVCI :

Niveau de Ressource	5h / 20 seq.	10h / 40 seq.	20h / 80 seq.	30h / 120 seq.
Niveau 1 (Contenus simples + quiz)	0,40 h	16 h	32 h	48 h
Niveau 2 (+25% activites interactives)	0,75 h	30 h	60 h	90 h
Niveau 3 (Serious games, simulations)	1,5 h	60 h	120 h	180 h

Les memes regles s'appliquent pour la mise a jour des ressources avec un coefficient de 0,5.

Module 6 - Tableaux de Bord et Statistiques

- Volume total d'heures par enseignant
- Repartition des activites pedagogiques par type
- Volume horaire par departement
- Enseignants ayant depasse leur charge statutaire
- Statistiques mensuelles

Module 7 - Generation des Etats et Exports

- Fiche individuelle enseignant
- Etat global des heures par periode
- Etat des paiements (heures complementaires)
- Export en format PDF et Excel

5. Architecture Technique

5.1 Vue Generale

L'application suit une architecture client-serveur en trois couches, avec une separation claire entre le frontend, le backend et la base de donnees. Un service de reconnaissance faciale est integre cote serveur pour securiser l'authentification.

Couche	Technologie	Role
Frontend	**Next.js (React 18+)**	Interface utilisateur, rendu SSR/CSR, appels API REST
Backend / API	**Python FastAPI**	Logique metier, calculs horaires, gestion des roles, API REST
Authentification faciale	**FaceAPI (Python)**	Reconnaissance faciale lors de la connexion, verification identite
Base de donnees	**SQLite (dev) / PostgreSQL (prod)**	Stockage relationnel des donnees applicatives
ORM	**SQLAlchemy + Alembic**	Abstraction base de donnees et migrations

5.2 Technologies et Outils

Frontend - Next.js

- Framework React avec rendu hybride (SSR + CSR)
- Gestion des routes avec le App Router de Next.js 14+
- UI : Tailwind CSS pour le style, Shadcn/ui ou Ant Design pour les composants
- Gestion d'etat : Zustand ou React Context API
- Appels API : Axios ou Fetch natif
- Generation de graphiques : Recharts ou Chart.js

Backend - Python FastAPI

- Framework web Python moderne, rapide et asynchrone
- Documentation API automatique avec Swagger UI
- Validation des donnees avec Pydantic
- Gestion de l'authentification JWT (JSON Web Tokens)
- Librairies : SQLAlchemy (ORM), Alembic (migrations), Passlib (hachage mots de passe)
- Export PDF : ReportLab ou WeasyPrint ; Export Excel : openpyxl

FaceAPI - Reconnaissance Faciale

- Integration dans le backend Python pour la verification biometrique
- Bibliotheque recommandee : face_recognition (dlib) ou DeepFace
- Flux : capture de la photo en frontend → envoi base64 → verification cote serveur
- Utilisation : etape de second facteur d'authentification apres le mot de passe
- Stockage des encodages faciaux en base de donnees (champ binaire ou JSON)

Base de Donnees

- SQLite en phase de developpement (aucune installation requise)
- Migration prevue vers PostgreSQL en production
- Gestion des migrations avec Alembic

6. Specifications Techniques Detaillees

6.1 Authentification et Securite

Le systeme implementera un mecanisme d'authentification en deux etapes :

Etape	Methode	Details
1	Login classique	Email + mot de passe hache (bcrypt), verification en base
2	Reconnaissance faciale (FaceAPI)	Capture webcam → encodage → comparaison avec le profil stocke
Session	JWT Token	Token d'acces signe avec expiration, stocke cote client

- Gestion des roles et permissions (RBAC : Role-Based Access Control)
- Protection des routes sensibles par middleware d'authentification
- HTTPS obligatoire en production
- Protection CSRF et validation des entrees

6.2 Structure de la Base de Donnees

Le modele de donnees comprend les entites principales suivantes :

Table	Champs Principaux
Utilisateur	id, email, mot_de_passe_hash, role, encodage_facial, created_at
Enseignant	id, nom, prenom, grade, statut, departement, taux_horaire, email, telephone, user_id
Cours	id, intitule, filiere, niveau, semestre, nb_heures, nb_credits
Ressource_Pedagogique	id, type, niveau_complexite (1/2/3), cours_id, enseignant_id, date_creation
Activite_Pedagogique	id, type (creation/mise_a_jour), ressource_id, enseignant_id, nb_sequences, volume_horaire_calcule, annee_academique
Annee_Academique	id, libelle, date_debut, date_fin, statut (active/cloturee)

6.3 API REST - Points d'Entree Principaux

Le backend FastAPI exposera les endpoints suivants organises par domaine :

Methode	Endpoint	Description
Auth		

POST	/auth/login	Authentification (email + mot de passe)
POST	/auth/face-verify	Verification par reconnaissance faciale (FaceAPI)
Enseignants		
GET	/enseignants	Liste des enseignants (avec filtres)
POST	/enseignants	Creer un nouvel enseignant
PUT	/enseignants/{id}	Modifier les informations d'un enseignant
DELETE	/enseignants/{id}	Supprimer un enseignant
Activites & Calculs		
POST	/activites	Saisir une activite pedagogique
GET	/volumes/{enseignant_id}	Calculer le volume horaire d'un enseignant
Exports		
GET	/exports/pdf/{id}	Fiche recapitulative enseignant en PDF
GET	/exports/excel	Etat global des heures en Excel

7. Exigences Non Fonctionnelles

7.1 Performance

- Temps de reponse API inferieur a 500ms pour les requetes courantes
- Interface utilisateur reactive et fluide (SPA avec Next.js)
- Traitement de la reconnaissance faciale en moins de 3 secondes

7.2 Disponibilite et Fiabilite

- Application accessible via tout navigateur web moderne (Chrome, Firefox, Edge)
- Interface responsive compatible desktop et tablette
- Sauvegarde automatique des donnees

7.3 Securite

- Authentification obligatoire pour acceder a toutes les fonctionnalites
- Donnees transmises via HTTPS en production
- Mots de passe haches avec bcrypt (facteur de cout >= 12)
- Encodages faciaux stockes de maniere securisee
- Journalisation des actions sensibles (audit log)

7.4 Maintenabilite

- Code structure selon les conventions de chaque framework
- Documentation du code (docstrings Python, JSDoc)
- Variables d'environnement pour les configurations sensibles

8. Organisation de l'Equipe et Planning

8.1 Repartition des Roles

Sur 2 semaines avec 5 membres, la repartition suivante est recommandee :

Membre	Role / Domaine	Responsabilites principales
Membre 1	Frontend (Next.js)	Pages login, dashboard admin, gestion enseignants, composants UI
Membre 2	Frontend (Next.js)	Pages cours, ressources, activites, espace enseignant, exports
Membre 3	Backend (FastAPI)	API enseignants, cours, ressources, authentification JWT, securite
Membre 4	Backend (FastAPI)	Calcul volumes horaires, activites, exports PDF/Excel, tableaux de bord
Membre 5	BDD + FaceAPI	Modele de donnees SQLite, migrations Alembic, integration FaceAPI, tests

8.2 Planning sur 2 Semaines

Phase	Taches	Responsables
Semaine 1 J1-J2	Mise en place environnement, structure projet, modele BDD, maquettes UI	Toute l'equipe
Semaine 1 J3-J5	Developpement modules : Authentification (login + FaceAPI), gestion enseignants, cours	Membres 1, 3, 5
Semaine 2 J6-J8	Developpement : activites pedagogiques, calcul horaires, tableaux de bord	Membres 2, 4
Semaine 2 J9-J10	Generation exports (PDF/Excel), tests d'integration, corrections, documentation	Toute l'equipe

9. Livrables Attendus

A la fin du projet, les livrables suivants devront etre fournis :

N	Livrable	Description
1	**Application web fonctionnelle**	Frontend Next.js + Backend FastAPI deployes et operationnels
2	**Base de donnees operationnelle**	Schema SQLite avec donnees de test et scripts de migration
3	**Code source complet**	Repository Git avec README, structure propre et commentee
4	**Documentation technique**	Architecture, API (Swagger), instructions d'installation
5	**Guide utilisateur**	Manuel d'utilisation pour chaque profil (Admin, Secretaire, Enseignant)
6	**Rapport final du projet**	Rapport academique avec contexte, architecture, choix techniques, bilan

10. Conclusion

Ce cahier des charges definit le cadre complet du projet "Application de Gestion des Heures des Enseignants" de l'UVCI. La combinaison des technologies Next.js, Python FastAPI, FaceAPI et SQLite permet de construire une solution moderne, securisee et evolutive, repondant aux besoins identifies.

L'equipe de 5 membres disposant de deux semaines devra respecter le planning etabli et assurer une communication continue pour garantir la coherence entre le frontend et le backend. Une attention particuliere devra etre portee a l'integration du module FaceAPI et au calcul automatique des volumes horaires, qui constituent les deux points techniques les plus specifiques du projet.

Ce document servira de reference tout au long du developpement et sera mis a jour si necessaire pour refleter les evolutions du projet.